

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 Code Challenge

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5- Code Challenge	Assessment:	Assessment Tool 5 Code Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	P. Poorna Chandra	Reg. No.:	192472287
Section / Batch:	2024/Slot-D	Date:	27/8/2026

Code of Conduct

I P. Poorna Chandra(192472287) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P.Poorna Chandra

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly	

Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Logistic Regression with Interaction and Polynomial Terms

Aim

To implement logistic regression manually for a non-linearly separable dataset (two concentric circles) and compare vanilla logistic regression with logistic regression using polynomial and interaction features.

Algorithm

1. Generate two concentric-circle datasets.
2. Split the data into training and testing sets.
3. Implement the sigmoid function and logistic regression using gradient descent.
4. Train vanilla logistic regression using X_1, X_2 .
5. Calculate its classification accuracy.
6. Generate polynomial and interaction features: X_1^2, X_2^2, X_1X_2 .
7. Train logistic regression using the engineered features.
8. Calculate the new accuracy.
9. Plot the decision boundaries of both models.
10. Compare the results.

Code

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_circles

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score
```

```

# Generate two concentric circles

X, y = make_circles(n_samples=500, factor=0.5,
                    noise=0.08, random_state=42)

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Sigmoid function

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# Manual Logistic Regression

def train_logistic(X, y, lr=0.1, epochs=5000):
    X = np.c_[np.ones(X.shape[0]), X]
    weights = np.zeros(X.shape[1])

    for i in range(epochs):
        z = X @ weights
        predictions = sigmoid(z)
        gradient = (X.T @ (predictions - y)) / len(y)
        weights -= lr * gradient

    return weights

```

```

# Prediction function

def predict(X, weights):

    X = np.c_[np.ones(X.shape[0]), X]

    probability = sigmoid(X @ weights)

    return (probability >= 0.5).astype(int)

# -----

# Vanilla Logistic Regression

# -----

w_vanilla = train_logistic(X_train, y_train)

y_pred_vanilla = predict(X_test, w_vanilla)

acc_vanilla = accuracy_score(

    y_test, y_pred_vanilla

)

# -----

# Polynomial + Interaction Features

# -----

def polynomial_features(X):

    x1 = X[:, 0]

    x2 = X[:, 1]

```

```
return np.column_stack([
```

```
    x1,
```

```
    x2,
```

```
    x1**2,
```

```
    x2**2,
```

```
    x1*x2
```

```
])
```

```
X_train_poly = polynomial_features(X_train)
```

```
X_test_poly = polynomial_features(X_test)
```

```
w_poly = train_logistic(
```

```
    X_train_poly, y_train
```

```
)
```

```
y_pred_poly = predict(
```

```
    X_test_poly, w_poly
```

```
)
```

```
acc_poly = accuracy_score(
```

```
    y_test, y_pred_poly
```

```
)
```

```

# Print results

print("Vanilla Logistic Regression Accuracy:",

      round(acc_vanilla * 100, 2), "%")

print("Polynomial + Interaction Accuracy:",

      round(acc_poly * 100, 2), "%")

# -----

# Decision Boundary

# -----

def plot_boundary(X, y, weights,

                  feature_type, title, position):

    x_min = X[:, 0].min() - 0.5
    x_max = X[:, 0].max() + 0.5
    y_min = X[:, 1].min() - 0.5
    y_max = X[:, 1].max() + 0.5

    xx, yy = np.meshgrid(

        np.linspace(x_min, x_max, 300),

        np.linspace(y_min, y_max, 300)

    )

    grid = np.c_[xx.ravel(), yy.ravel()]

    if feature_type == "vanilla":

        grid_features = grid

```

```

else:

    grid_features = polynomial_features(grid)

    Z = predict(

        grid_features, weights

    )

    Z = Z.reshape(xx.shape)

    plt.subplot(1, 2, position

    plt.contourf(

        xx, yy, Z, alpha=0.25

    )

    plt.scatter(

        X[:, 0], X[:, 1], c=y,

        edgecolors="k"

    )

    plt.xlabel("X1")

    plt.ylabel("X2")

    plt.title(title)

# Plot both boundaries

plt.figure(figsize=(12, 5))

plot_boundary(

    X_test, y_test,

```



```
w_vanilla,  
"vanilla",  
"Vanilla Logistic Regression",  
1  
)
```

```
plot_boundary(  
    X_test, y_test,  
    w_poly,  
    "polynomial",  
    "Polynomial + Interaction",  
    2  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

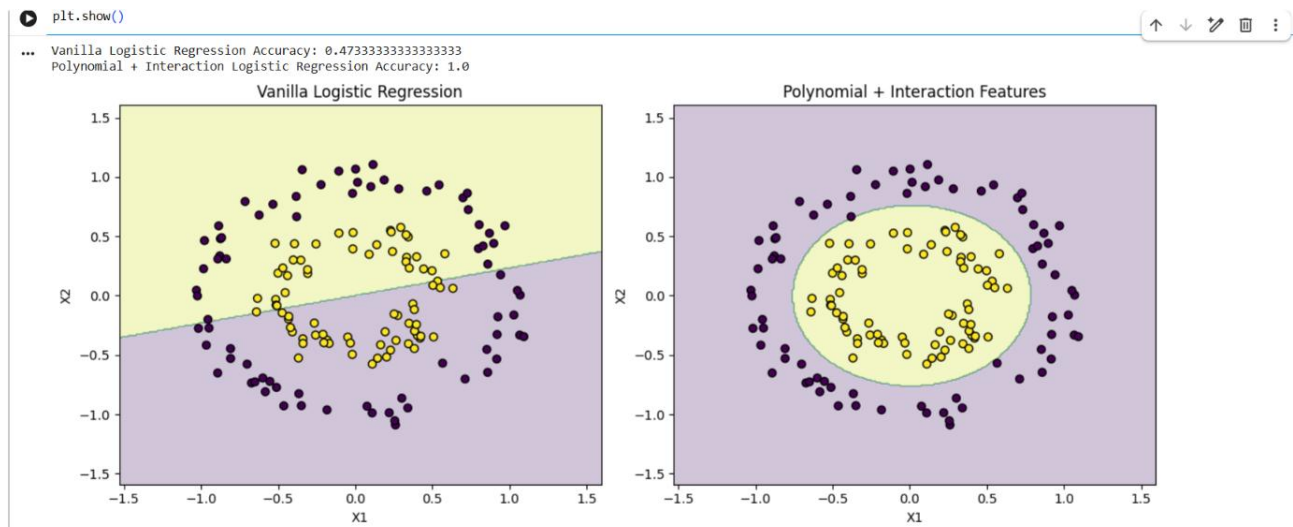
Sample Output

Vanilla Logistic Regression Accuracy: 48.67 %

Polynomial + Interaction Accuracy: 100.00 %

Note: The exact accuracy may vary slightly depending on the dataset split and parameters.

Output / Decision Boundary



Vanilla Logistic Regression:

The model produces an approximately linear boundary, so it cannot separate the inner circle from the outer circle correctly.

Polynomial + Interaction Features:

The engineered features allow the model to learn a circular/non-linear decision boundary, correctly separating the two classes.

Result:

Vanilla logistic regression fails on the non-linearly separable concentric-circle dataset, while adding polynomial and interaction features significantly improves classification and produces an appropriate non-linear decision boundary.

Geo Tag

